

lic Slack channel for communication around the incident like release-post-13.8-deploy-failure. That channel is then shared with release-post for others to follow along.

5. Begin a Zoom call. Post the invitation to the Zoom room in the newly created Slack channel.

6. Technical Advisor assumes control from the Release Post Manager.

7. Investigation begins. Be as visible as possible, share your screen. Consider recording the Zoom session.

8. Action taken.

9. Blocker is resolved.

10. Close Zoom.

11. Collect screenshots, recordings, terminal history, comments in Slack, or other evidence on the issue.

12. Open a new issue to document the incident, deposit history, and add suggestions for corrective action or prevention. Link issue to the current release post retrospective.

See also: Google SRE Ch. 14

Managing ongoing technical support tasks

Anyone can contribute to technical issues that support the Release Post Process with the burden being mostly on the current volunteer tech advisor. The following outlines how to manage active and upcoming issues.

Tracking

Use the ~Release Post::Tech Advisor labels for issues that require changes to the tools that facilitate the delivery of a release post. Create issues for lower priority challenges that arise during the milestone or as an artifact of decisions made during a retrospective. Creating an issue for immediate incident response isn't required since the delivery of the release post requires quick resolution and synchronous communication.

This board organizes these labeled issues into a familiar software development workflow. When working on an issue, assign yourself and strive to keep the issue up-to-date with the proper workflow label and weekly async updates. Technical Advisors are welcome to make recommendations and apply milestones to the issues.

Handoff

It's unlikely that one technical advisor will serve in back-to-back milestones. Therefore, clearly communicating with the incoming technical advisor about the state of issues as part of release post retrospective and kickoff is a good idea. To do so:

- create a transition issue
- recommend actions, prioritization, and milestones changes for any issues you are actively working
- unassign them from yourself and change the workflow label if you don't plan on working on so it's clear they need to be picked up
- after your final rotation on the release post, reach out to the next tech advisor for a coffee chat to provide them with helpful information about any issues or bugs that are a priority for their upcoming cycle

Should you prefer to continue to contribute to an issue under active development after your volunteer rotation, that's great. In that situation, make it clear through assignments and issue updates that you will be the DRI.

extras:

- title: "Example title"
description: | supports markdown
Description

Multiple blocks:

yaml

extras:

- title: "Example title one"
description: | supports markdown
Description one
- title: "Example title two"
description: | supports markdown
Description two

Apply the following labels to the MR:

- release post item
- release post
- release post item::extras

For review, select the technical writer assigned to the stage.

When the MR is approved, add the Ready label before merging.

Deprecations, removals, and breaking changes

Deprecation, removal, and breaking change announcements appear in GitLab Docs and in the release post of the announcement's corresponding milestone.

Before making an announcement, review the breaking changes, deprecations and removals guidance to ensure you:

- Are minimizing disruption for our customers.
- Are providing the required advance notice for workflow changes.
- Have determined if the change is a breaking change.

Milestone due dates

- Thursday, 3 weeks before release: Announcement MR has been created
- Thursday, 1 week before release: MR has been assigned to a technical writer
- Thursday, 1 week before release: MR has been merged

Walkthrough video

This video will walk you through the process of making an announcement:

<figure class="videocontainer">

<iframe src="https://www.youtube.com/embed/9gy7tg94j7s" title="Overview of deprecation and removal of features in GitLab releases" frameborder="0" allowfullscreen="true"> </iframe>

</figure>

Deprecations and other planned breaking change announcements

- To be added by Product Managers or Engineering Managers and merged by Technical Writers at least 3 milestones ahead of the planned removal date.

For example, if the intended removal milestone is 17.0, given the following release schedule: 16.9, 16.10, 16.11, 17.0, then 16.9 is the third milestone preceding intended removal, and the last milestone for deprecation announcements.

- Create a separate MR for each announcement.
- Do not edit the features.yml file until the feature has been removed from the product, or the breaking change has been implemented.
- If you want to bundle multiple announcements in one MR, for example if it's a group of dependent deprecations that will happen on the same date as "all or none," reach out and first discuss this with the Release Post Manager.

Creating the announcement

1. Create a new branch in the gitlab-org/gitlab project.
 1. Copy the template file and save it in the data/deprecations folder.
 1. Name the file XX-YY-feature-name.yml, where XX-YY is the milestone of the initial announcement. For example, 14-7-pseudonymizer-deprecation.yml.
 1. Create a merge request using the Deprecations MR template for the change description.
 1. The title must clearly explain the deprecation or planned change. For example:
 - "The confidential field for a Note is deprecated."
 - "The maximum number of characters in a job name will be limited to 250."
 - "Access tokens with no expiration date will be changed to have an expiration of one year."
 - "The omniauthcrowd gem is deprecated."
 1. The description must:
 - Be clear and concise.
 - Give a brief explanation of the details or reasons for the change.
 1. The description must also explain what the user must do as a result of the change. In other words, the entry must be actionable by users or admins. For example:
 - "Use the internal keyword instead of confidential."
 - "Reduce the number of characters in all job names to be 250 characters or less."
 - "Give an expiration date to any access tokens that have no expiration date."
 - "Stop using the omniauthcrowd gem. It will be removed and will not be replaced."
 1. Avoid announcements like "Feature A is deprecated and will be removed. Wait until the replacement feature is released, and switch to it at that time."
- If the entry is not actionable, delay the deprecation or change announcement until there is an action available for users to address the change.

1. Assign reviewers as recommended in the template.

1. Set the `breakingchange` value to true and add the ~"breaking change" label to the MR. If the deprecation or planned change will not cause a breaking change (rare, but possible), use false and do not add the label.

1. Assign the MR to the technical writer assigned to the stage.

Reviewing and merging the announcement

1. The TW Reviewer reviews the content, adds a commit that updates the deprecations doc, and merges the MR by the Thursday, 1 week before release. After merging, the announcement will be visible on the deprecations documentation page within an hour.

1. If the MR is at risk of missing the cut off date, open a duplicate MR and set the target branch to X-Y-stable-ee where X-Y aligns with the version released X.Y. If you have trouble, ask for help in mr-buddies or refer to the full process for backporting an MR.

Announcing an End of Support period

The deprecation template provides an option to end support for a feature prior to its removal. This option should only be used in special circumstances and is not recommended for general use. Most features should be deprecated and then removed.

An End of Support milestone must be at least 3 milestones after the deprecation announcement. For example, if the deprecation announcement is made in 15.1, the End of Support milestone must be in 15.4 at the earliest. There is no requirement for the gap between the End of Support milestone and the Removal milestone.

If an End of Support milestone is announced, it will be displayed under the title of the deprecation announcement on the Deprecations page. End of Support milestones are not currently displayed in the release post.

When to define an End of Support period

- If there is a long period of time between the deprecation and the planned removal.
- If supporting the feature during an extended deprecation period would affect engineering velocity (bloat, dependency management, etc.).

Communicating End of Support

If you decide to declare an End of Support period:

- Check for any Support Stable Counterpart (also listed on the product categories page) for your development group and tag them in the MR that adds a value to the `endofsupportmilestone`.
- If your group does not have a Support Stable Counterpart, look for a stage or section Support Counterpart. If none, please post in the supportleadership Slack channel with a link to the readiness issue (next line).
- Please also open a Support Readiness issue following the Support communications guidance.

Editing an announcement entry

This process is very similar to creating an announcement entry, with the difference being that

the YAML file already exists.

To edit an existing entry:

1. Create a new branch in the gitlab-org/gitlab project.
1. Find and edit the .yml file in the data/deprecations directory.
1. Create a merge request for the change, and use the Deprecations MR template for your change.
1. Assign reviewers as recommended in the template.
1. Assign the MR to the technical writer assigned to the stage. (By the Thursday, 1 week before release if revising an entry for the upcoming release)
1. The TW Reviewer reviews the content, adds a commit that updates the docs, and merges the MR. (By the Thursday, 1 week before release)

Patch and Security release posts

The Delivery team is responsible for creating release posts for patch and security releases.

Release posts should live in sites/uncategorized/source/releases/posts. For patch and security releases, please make sure to specify them in the title, add the correct category:

- Patch releases:
 - title: "GitLab Patch Release: x.y.z and x.y.z"
 - categories: releases
- Security releases:
 - title: "GitLab Security Release: x.y.z and x.y.z"
 - categories: releases

Creating an MR for What's New entries

Video walkthrough of the process

"What's new" can be seen by clicking on the ? icon in the navigation menu of GitLab and choosing "What's new."

The What's New MR will be initiated by the Release Post Manager on the Tuesday of release week, finalized on the Wednesday of release week, and typically get merged by a maintainer 2 to 4 hours AFTER the release post is live on the release date. The exact timing of the merge depends on the availability of a maintainer to merge it

1. On the Tuesday of release week, the Release Post Manager creates the What's New MR.
1. The RPM will pull all of the final/approved primary items in the release post to feature in "What's New"
 - The selected items can but do not have to align with the primary features of the release post. If the RPM feels comfortable making a unique selection, they may do so.
 - It is recommended to strive for a balance of features available to both free and paid users. If we only include Ultimate features, non-Ultimate users may stop engaging.
1. The Release Post Manager will create a new MR in the gitlab.com/gitlab-org/gitlab project

1. Use this template to create a new file in the gitlab/data/whatsnew directory.
1. Title the file as YYYYMMDD0001XXYY.yml - for example, the 13.4 entry is titled 2020093000011304.yml.
1. Using the content that is in the release post items, duplicate the content for What's New.
1. Sometimes it is necessary to trim down the description. Keep in mind that these should be short, and generally a single paragraph.
1. SaaS-only features need to use [free, premium, ultimate] in the What's New MR instead of [free, silver, gold] or core as is used in the release post items. We will streamline this discrepancy in the future, but for now, the RPM should update the values as necessary when creating the What's New MR.
1. Stage names are case-sensitive, so be sure that the stages are capitalized in order for the pipeline to pass.
1. Images are not required, but encouraged when possible.
1. For videos, you will need to provide an image. For YouTube videos, you can use the image URL, which can be found by inserting the youtube unique ID into this format: <https://img.youtube.com/vi/insert-youtube-video-id-here/hqdefault.jpg>. For cases where a video thumbnail doesn't look great, consider using a generic image from [the source/images/growth directory].
1. Images will end up at a URL like <https://about.gitlab.com/images/XY/XXXXXXX.XXX>. Make sure you provide a full URL for the YAML entry. Ex: <https://about.gitlab.com/images/137/reviewerssidebar.png>.
1. If an image is not available, you can use a generic image (ex: <https://about.gitlab.com/images/ci/gitlab-ci-cd-logo2x.png>) or omit the imageurl.
1. Before committing the MR, check the YAML with a validator.
1. Add the ~"documentation" label.
1. Apply the affiliated release milestone (it's ok if it says "expired") and labels whats new and release post.
1. On the Tuesday of release week, when the MR is ready for review, assign @name of PLT member who is reviewing this month] as reviewer and @mention them in the MR to complete their review by the Wednesday of release week. The name of the PLT member who is reviewing this month can be found on the [release post scheduling page]
1. After the release post is live and you have verified the images load locally in GDK by pulling down the What's New branch, have the MR reviewed following our standard code review process and have it merged by a maintainer. It is recommended to communicate directly to the maintainer that the MR is time sensitive to avoid unnecessary delays.
1. Open a duplicate MR and set the target branch to X-Y-stable-ee where X-Y aligns with the version released X.Y. Assign it to the same maintainer that merged the previous MR. This second MR ensures that any additional releases to version X.Y include this "What's New" update. If you have trouble, ask for help in mr-buddies or refer to the full process for backporting an MR.

IMPORTANT: The MR should not be merged until after the release post is live on the release date or the images will not display. After the release post is live, but before merging, the branch should be checked out and the content checked in GDK to make sure that all images are displaying, links are accurate, and that the What's New items are part of the final release post. Only once those are confirmed should the MR be merged. Typically this means the What's New content will be live on the day or two after the release date, depending on maintainer reviews.

Related

Pages

- GitLab the product
- General guidance on deprecations, removals and breaking changes
- Release Post volunteer schedule
- Security and monthly releases
- Features per release
- GitLab CE and GitLab EE changelog
- Release managers

Templates

- Monthly release post MR template
- Release post item MR template
- Deprecation issue template
- Deprecation MR template
- Release Post retrospective issue template
- YAML content block samples
- Patch release template
- Security release template

SECTION: marketing/blog/release-posts/managers.md

Each month a Product Manager will lead the release post authorship, and a Product Marketing Manager will lead the messaging. Other members will be responsible for the structural check, copy edit, and social media posts. For each monthly release, there are also technical Release Managers who support the effort.

<!-- To update the list, see the relevant YAML file <https://gitlab.com/gitlab-com/www-gitlab-com/-/blob/master/data/releasepostmanagers.yml> -->

{{< marketing/release-post-scheduling >}}

SECTION: marketing/blog/release-posts/manual-release-post-kickoff/index.md

Manually create your release post branch and required directories/files

If the automated pipeline fails, the manual steps below can be done either locally or using the GitLab Web IDE:

1. On gitlab.com/gitlab-com/www-gitlab-com create a new branch release-X-Y from master.
1. On release-X-Y branch, create the blog post file, containing the introduction and the blog post frontmatter information:
 1. In `sites/uncategorized/source/releases/posts/` directory, add a new file called YYYY-MM-22-gitlab-X-Y-released.html.md by copying the monthly release blog template.

1. Add the release number to sites/uncategorized/source/releases/posts/YYYY-MM-22-gitlab-X-Y-release.html.md on line 36 and make sure to remove the backticks.

1. Add your name as the author to sites/uncategorized/source/releases/posts/YYYY-MM-22-gitlab-X-Y-release.html.md on lines 3 and 4

1. Update the "which includes our YY release kickoff video" line replacing the YY (including removing the backticks) to reference the next release in sites/uncategorized/source/releases/posts/YYYY-MM-22-gitlab-X-Y-release.html.md on line 41

1. On release-X-Y branch, create the release post data directory, to which features and other data will be added:

1. Create a new directory XY in the data/releaseposts directory.

1. Copy data/releaseposts/unreleased/samples/mvp.yml into data/releaseposts/XY/mvp.yml.

Create the release post MR

Create a merge request with the introductory changes after the previous post has been merged and before the feature freeze date to make the post available to receive contributions from the team:

1. The source branch must be release-X-Y and the target branch master.

1. Set the title to Draft: Release post - GitLab X.Y. Prefix the title with Draft:.

1. Confirm that "Delete source branch when merge request is accepted" is selected.

1. Use the release post template for your MR.

Now that you have created the release post MR, refer to the checklist in the MR for each action that you need to take and the due dates of each action. Keep in mind the MRs for usability improvements, bugs, and performance improvements have their own checklists to be completed, including a task for the Release Post Manager to merge these MR by the 17th prior to final content assembly.

Create Retrospective Issue

1. Create a release post retrospective issue by using the Release post retrospective template, and use Release Post X.Y Retrospective as a title.

1. Add the appropriate milestone to the issue.

1. Assign yourself to the issue.

Note: After you have created the release post MR and all the related files, refer to the checklist in the MR for each action that you need to take and the due dates of each action. Keep in mind the MRs for usability improvements, bugs, and performance improvements have their own checklists to be completed, including a task for the Release Post Manager to merge these MR by the 17th prior to final content assembly.

Content assembly: merging release post items (content blocks) to your branch

{{% alert title="Important" color="info" %}}

This procedure applies until the 18th, at <time datetime="07:59">7:59 am UTC (18th 2:59 am ET / 17th 11:59 pm PT)</time>. After this time, anyone who wants to include a change in the upcoming release post can either coordinate updates directly on the release post branch with the Release

Post Manager or submit it in a separate MR, targeting the release-X-Y branch, and assign it to the Release Post Manager to merge. For more information, see our documentation on how to Develop on a feature branch.

{{% /alert %}}

When it is time to assemble the release post, this will be done by moving the content block files from data/releaseposts/unreleased to data/releaseposts/XY, and images from source/images/unreleased to source/images/XY.

Those block items comprise of the release post items that each PM creates for each feature.

The bin/release-post-assemble script makes this easy to do:

```
bash
git checkout master
git pull
git checkout release-X-Y
git pull
git merge master
bin/release-post-assemble
git status
confirm that content blocks and images have moved from unreleased to XY
git add .
git commit -m "Content assembly"
git push origin release-x-y
```

If the content assembly bot fails

If for some reason the content assembly bot fails, the simplest way to move forward is to move the files manually. There is also a video walking through the changes [here](#).

1. Manually move all the .yaml files from /data/releasesposts/unreleased/ to /data/releaseposts/xy/ (xy being the release post directory e.g. 132). | Note: Leave the /samples directory in the same location, don't move it.

1. Manually move all the images in /source/images/unreleased/ to /source/images/xy/.

1. Using a text editor like VS Code find and replace all the image paths under imageUrl: in each release post .yaml file from /unreleased/ to /xy/. The video above demonstrates that.

1. git commit and git push and you should be good to go.

Release post assembly

The release post assembly script moves release post content blocks and their images to the current release directory.

It uses a simple regexp to locate content files and images. It performs no validation. In the future, it would be simple to combine the functionality with the linter to reduce the number of scripts to maintain.

In preparation for content assembly on the 18th of the month, the Release Post Manager should ensure their local dev environment is up to date (e.g., running latest version of Ruby). Follow the steps in the "Content assembly and initial review" section of the release post MR checklist to prepare a local dev environment in advance.

Possible Script Errors with Corrective Actions

The development.md contains steps to setup your local development environment.

Here are some of the common errors that a user might encounter where it may not be clear as what to do.

You are missing a required Ruby Gem

You might receive obscure error such as this:

```
bash
```

```
Traceback (most recent call last):
```

```
6: from ./bin/release-post-item:5:in `<main>'
5: from ./bin/release-post-item:5:in `require_relative'
4: from /Users/chase/work/www-gitlab-com/lib/releaseposts.rb:13:in `<top (required)>'
3: from /Users/chase/work/www-gitlab-com/lib/releaseposts.rb:13:in `require_relative'
2: from /Users/chase/work/www-gitlab-com/lib/releaseposts/postentry.rb:6:in `<top (required)>'
1: from
/Users/chase/.asdf/installs/ruby/2.6.6/lib/ruby/2.6.0/rubygems/coreext/kernelrequire.rb:117:in
`require'
/Users/chase/.asdf/installs/ruby/2.6.6/lib/ruby/2.6.0/rubygems/coreext/kernelrequire.rb:117:in
`require': cannot load such file -- styledyaml (LoadError)
```

In this case, Ruby is trying to load a file named styledyaml. It's not clear that this is a gem (a self-contained Ruby library), but the require statement in the output is a clue that there is some unresolved dependency here. The action you should take in this case is to run bundle install. You can also run ./bin/doctor and it should provide guidance on what to do. If you're uncomfortable or encounter have difficulty here, you can reach out to the release post DRI for advisement.

Ruby mismatch

If you have a Ruby version manager installed, you may receive an error in your terminal along the lines of ruby 3.0.0 Not installed. Run "asdf install ruby 3.0.0"

It's possible that your Ruby version is out of date with what is required to run handbook scripts. You should be able to run ./bin/doctor to compare your current Ruby version with that in the .tool-versions file.

The action you can take is to install the required Ruby version

To install Ruby in the most popular Ruby version managers, try:

- For asdf, run the following: `asdf install ruby 3.0.0`
- For rbenv, run the following: `brew upgrade rbenv ruby-build && rbenv install 3.0.0`
- For rvm, run the following: `rvm install 3.0.0`

If you're uncomfortable or encounter have difficulty here, you can reach out to the release post DRI for advisement.

Note that the handbook currently suggests rvm, while engineering has adopted asdf. You may find other references to rbenv in this documentation too. Any of these are fine, but they all work a bit differently and you only need one Ruby version manager installed.

It is also possible that your ruby version manager is misconfigured or your settings have been altered because of an upgrade to macOS especially from earlier versions to Catalina or higher. It's difficult to suggest an action for this scenario, you may want to reach out to the release post DRI for advisement.

Gems install correctly, but you still have a missing gem error

The ruby gem package manager is called bundler. Depending on the version of bundler you have installed, it is possible to configure bundler to install gems in a location different from the usual (and required) location by passing the `--path thatotherdirectory` are remembered between invocations and will be stored in `./bundle/config` or in `./bundle/config`.

If you look in the `./bundle/config` file you might see:

```
yaml
BUNDLEPATH: "thatotherdirectory"
```

The action you can take here is to edit that file `./bundle/config` and possibly `./bundle/config` to remove the BUNDLEPATH setting and re-run `bundle install`. You may also want to remove the `thatotherdirectory` which is often `vendor`. If you're uncomfortable or encounter have difficulty here, you can reach out to the release post DRI for advisement.

Locking support

You might encounter a message like this about locking support when you push a local commit to origin.

```
bash
Locking support detected on remote "origin". Consider enabling it with:
$ git config lfs.https://work-gitlab/gitlab-com/www-gitlab-com.git/info/lfs.locksverify true
```

You can probably safely ignore this suggestion. More documentation on Git LFS file locking.

JAMF and git-lfs conflict

In the process of trying to push your commits to gitlab.com git is trying to verify the SSL cert. If you have JAMF installed (and you should for compliance reasons), git might find a

different certificate for gitlab.com and throw an error about `Post "https://gitlab.com/gitlab-com/www-gitlab-com.git/info/lfs/locks/verify": x509: certificate signed by unknown authority error: failed to push some refs to 'gitlab.com:gitlab-com/www-gitlab-com.git'`.

The action you can take here is to contact IT

More information can be found in this issue.

SECTION: marketing/brand-and-product-marketing/_index.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

<i class="fas fa-bullhorn fa-fw"></i> Welcome to the GitLab Brand and Product Marketing Handbook

The Brand and Product Marketing team's goal is to accelerate GitLab's path to market.

- We tell the GitLab story at the brand, corporate, solution, and product levels.
- We develop market insights, leadership positioning, messaging, and narratives that elevate our brand, show value in our solutions, and fuel demand for our platform.
- We activate brand ambassadors and customer advocates to show GitLab's momentum and trust in the market.

Team Disciplines · What We Do, Why We Do It

Brand

Brand: We create awareness, trust, and affinity for the GitLab brand.

We develop umbrella, corporate-level messaging to establish GitLab as market and category leader. We conduct research to define, evolve, and measure the health of the GitLab brand. We provide brand stewardship and governance, empowering internal and external advocates to become brand champions.

Product and Solution Marketing

Product and Solution Marketing: We develop insights, positioning, and messaging that communicate the value of our platforms and solutions.

Product and Solution Marketing consists of several specialities including core product marketing, customer advocacy, market insights, analyst relations, and competitive intelligence.

We create positioning and messaging to fuel demand and adoption of GitLab's solutions and products. We develop market and competitive intelligence in order to develop actionable insights on market trends, customer needs, and competitive insights that guide go-to-market and product strategy. We identify, recruit, and activate customer advocates to demonstrate momentum and trust in the market.

Design

Design - Deliver creative designs and experiences that elevate our brand and fuel demand.

We develop our brand visual identity design to ensure it stays relevant in the market and reflects the benefits and quality of our product. We are the creative partners for GitLab marketing. We create, conceptualize, and design high quality brand experiences.

Content

Content - Showcase our leadership through easy-to-find content that is educational, thought-provoking, and market-defining.

We create, review, and improve content to establish GitLab as the leader in modern software development and DevOps, while also sharing the company's vision for the future. We create content that maintains and improves our search rankings around key terms. We create content for go-to-market campaigns that drive awareness, growth, engagement, and conversion.

Our core capabilities

The Brand & Product Marketing team operates at the intersection of marketing, sales, customer success, and product management. As a team, we measure our impact through the following capabilities:

SECTION: marketing/brand-and-product-marketing/design.md

<link rel="stylesheet" type="text/css" href="/stylesheets/biztech.css" />

<i class="fas fa-bullhorn fa-fw"></i> Welcome to the GitLab Brand Creative Handbook

We develop our brand visual identity design to ensure it stays relevant in the market and reflects the benefits and quality of our product. We are the creative partners for GitLab marketing. We create, conceptualize, and design high quality brand experiences.

Overview

Purpose

Why we exist

As stewards of the GitLab brand, our goal is to educate and enable the wider organization with resources to effectively and honestly communicate what the company does to our internal and external audiences.

Vision

Where we're going

The GitLab Brand Design and Video teams will elevate the brand beyond the logo and visuals - positioning ourselves as experts in brand strategy and behavior (how the brand presents itself, how it's perceived, and what makes it authentic)

Mission

What we do

Create simple, effective, and intentional brand experiences by solving complex problems; defining the what, why, and how, resulting in a message that's easy to understand.

Requesting Support

Please fill out one of these issue templates to request support. Please note, if these are not filled out we won't have the proper information for us to support your request.

For more information on the brand video process, you can visit the brand video team handbook page.

- For more information, please see this announcement on Slack [HERE](#)

Brand Design & Video Issue Templates

1. Blog Image Request

Need an image for a blog? Please use this template to request blog images.

2. Brand Review Request

Need a brand review? Please use this template to request a brand or design review of multiple or single assets.

- For brand review only - do NOT use this template for requesting new assets or designs.
- Do NOT use any of the below issues for brand review.

3. Brand Review Request of Localized Assets

Need a brand review of a localized asset? Please use this template to request a brand or design review of multiple or single localized assets.

- For brand review only - do NOT use this template for requesting new assets or designs.
- For non-English design or video assets only
- Do NOT use any of the below issues for brand review.

4. Content Design Request

Please use this issue request template for net new creative or refreshes needed for the following asset types:

- eBook / Solution Briefs
- Infographic / Diagrams
- One-Pager / Two-Pager
- Quarterly Update of Executive Candidate Info Packet
- Surveys
- Whitepaper
- Other but within the same category of assets

5. Event Asset Request

Please use this issue template when requesting net new or refreshing any of the following event-related assets:

- Event Booth
- Event Signage
- Event Social Posts

Do NOT use this template for For Event Swag + Giveaways. INSTEAD, please use the Swag Request issue template.

6. Illustration + Iconography Requests

Please use this issue template for any illustration and iconography needs.

7. Presentation Request

Please use this issue request template for updating existing presentation decks or when requesting design for net new decks.

8. Social Asset Request

Please use this issue template for net new or refreshing organic and/or paid social requests.

9. Swag Request

This issue template should be used for all custom swag requests including swag for TMRG programs, event swag + giveaways, etc.

10. Tanuki Tab Request for Team Expressions

Please use this issue template for custom badge or assets for your TMRG, Department or special project. Be sure to review the GitLab Team Expressions Style Guide, as this will be the system used for your Tanuki Tab request.

11. General Design Request

If your design request does not fall within one of the above categories, or if you need a Zoom background, please use this issue template to request a new design of multiple or a single asset.

- Do NOT use this template for brand reviews.

12. New Brand Video Request

Please use this issue when requesting a net new video.

13. Upload Video Request

Please use this issue template when requesting to have a video uploaded to Vimeo, YouTube, or another similar platform.

14. Video Edit Request

Please use this issue when requesting an update to an existing video or edits to video footage such as adding lower thirds, bumpers or slides to a zoom recording.

Request Instructions and Tips:

- When submitting your request, please title the request "CREATIVE REQUEST: [Descriptive name]"
- Please complete the issue form to the best of your ability. The more information our team can receive upfront, the quicker we can get started on your request.
- Please note, all requests need a minimum of a two-week turnaround, starting from the date we receive ALL the necessary information to begin the design, not from the date of your request submission. Please note that we cannot guarantee meeting your deadline if it falls under this two-week window.
- If the deadline for your request is under a two-week window, please ping our Senior Creative Operations Manager in Slack and include a link to your request, the deadline date, and why this is a quick-turn ask.
- When inputting your project deadline, please be mindful of time zone differences. For example, if you're operating a day ahead of our Brand Design Team (PT / GMT-8 time zone) and need deliverables completed by Tuesday, Feb 20, please select Monday, Feb 19 as your deadline to accommodate for the time difference.
- If you have any questions, comments, or concerns regarding your creative request, please reach out to our Senior Creative Operations Manager.

Contacting the team

The best way to get in contact with the Creative team is to fill out one of the above issue templates with your request, or to:

- Use the @gl-design tag in GitLab issues and epics for visibility.
- @-mention individuals from the team in GitLab issues and epics.
- Ask your question in the brand Slack channel.
- For video specific questions or assistance, please reach out in the brandvideo Slack channel.

Working with the brand

To learn more about our Brand Guidelines, self-service resources and assets, and training materials, check out the Brand Resources handbook. Below you will find more details about working with our brand and creative materials.

Providing feedback

The Brand Creative team is comprised of team members who all bring a variety of visual skills, knowledge, and brand expertise to each project. Brand awareness, visual consistency, and business value are the key factors we use to make creative decisions - all while we adhere to and build upon the systems outlined in our brand guidelines. When working with our team, it is important to provide constructive feedback by keeping these same factors in mind.

Constructive feedback takes into consideration our values of collaboration, efficiency, and results. The following tools can help keep feedback focused, all while allowing a project to move along in a timely manner:

- When requesting edits, please use objective feedback that keeps brand awareness, visual consistency, and business value in mind. Here are some examples of objective · vs. subjective · feedback.
 - · Industry data shows that ads with darker backgrounds garner more traffic; can we please update the color to our Purple 02p?
 - · The background color is throwing off the design for me. I saw an ad with magenta colors and liked that, can we do that instead?
- Final approvals and feedback should be communicated by one DRI throughout the lifespan of the project.
 - Note: It is most helpful if the DRI can consolidate feedback amongst any other involved team members before sharing back in the issue.
- To keep projects on track, there will be up to 3 rounds of creative drafts for review.
 - Note: Anytime a draft is shared, it has already gone through a number of iterations and reviews internally within the Brand Creative team, ensuring the best possible version is being presented with stakeholders.
- While outside examples and inspiration are encouraged to help with concepting, it is important to acknowledge that we cannot replicate others' designs - especially at the expense of our own brand identity.
- When reviewing work, it is the stakeholder(s) · responsibility to review for content, including grammar, punctuation, and spelling.

Stock photography

When it comes to our brand, photography is a powerful medium that can better contextualize and humanize the complex stories we tell. Our team has gathered approved images in our photo library that are ready-to-use. If you need images beyond what are in that library (this will be common until the library becomes larger over time), the Brand Design team has a license to download unlimited images from Adobe Stock (Note: Premium images are not included in our license).

Review our photography guidelines before searching for new images. For approval, download, and questions, contact our team in Slack or the relevant issue.

Partnership with third parties

In certain cases, the help of a third party agency or design partner may be brought in for a project. The following serves as criteria for when to outsource design:

- Smaller-scale projects, such as stickers or requests that do not meet our current business prioritization, where the brand guidelines provide sufficient creative direction and parameters for the third party to work with.
- Larger-scale projects where the Brand and Digital team need additional support given the timeline and/or scale of the request.

Whenever a third party is brought in to support design, the work must be shared with the Brand Design team to ensure brand integrity and that we are working transparently with one another.

Preserving the tanuki

We understand that customized versions of the tanuki have been meaningful to our community and represent important aspects of our culture and values. When we rebranded in 2022, we made the strategic decision to no longer alter the tanuki. These parameters are reflected in our Brand Guidelines and are an intentional pivot from our previous logo, which we often customized with different colors and accessories.

Why we maintain one look for our tanuki:

- We built an alternative. In 2024, we launched the Team Expressions Tanuki Tabs to specifically address the community's need for creative expression in a way that still preserves our logo. This scalable system empowers the community to express their vibrant groups · including community groups, geo locations, business functions, and TMRGs · without altering our logo.
- Consistency strengthens our brand. Our tanuki is our most valuable brand asset. Any alterations dilute our brand identity and reduce recognition as we continue establishing our market presence.
- We're building our brand maturity and an enterprise focus. Restyling our tanuki undermines its visual role in our brand, which conflicts with our brand strategy.
- Strategic resource management drives results. Our small Brand Design team supports the growing creative needs of the entire company, prioritizing work that aligns with business goals and drives clear business value. Custom logo requests multiplied beyond our team's capacity and don't align with our strategic priorities.
- Brand integrity extends everywhere. Even "internal-only" designs frequently spread beyond their intended use, which can compromise brand awareness. Our logo is often someone's first impression of our company; showing up consistently maintains a clear, trustworthy image that meets industry standards and is a necessity in the years following a rebrand.

Mascots

We have taken a strategic approach to refrain from using or creating mascots inspired by the tanuki. This stance maintains brand authenticity, a focus on results for customers, and builds deeper trust with our community who use GitLab daily to create software that powers our world. The framework below outlines our vision for how our brand comes to life without a mascot character:

- Strategic differentiation from competitors: Our key competitors have taken an approach to make their mascots synonymous with their brand. By forgoing a mascot, GitLab establishes a distinct brand identity that doesn't compete in the same playful, character-driven space, which helps differentiate us from our biggest competitors.

- Brand maturity and enterprise focus: GitLab's focus on professional software development and enterprise DevSecOps aligns with a more sophisticated brand that focuses on business value, as opposed to character-driven marketing.
- Leveraging the tanuki as a brand element, not a mascot: The tanuki logomark is thoughtfully integrated as a symbolic element of our brand that represents GitLab's values. Centering DevSecOps at the core of its construction makes the tanuki a tactical brand device representing the limitless possibilities of our platform, and should be elevated as such.
- Avoiding mascot pitfalls: Our competitor's own brand guidelines acknowledge that mascots "work best internally" and specifically advise against their use for topics like money, security, sales, and enterprise offerings." By preserving the tanuki as a logomark rather than an anthropomorphized character, we are able to generate positive business impact and trust with our brand.

We believe there are alternative ways to elicit delightful engagement between our brand and communities, without relying on a mascot. These include, but are not limited to:

- User-generated content and community stories
- Behind-the-scenes content showcasing real GitLab team members
- Educational content that engages with users and provides value
- Interactive experiences and tools through brand activations
- Celebration of customer successes and innovations

Any future reconsideration of this approach would require a fundamental shift in our marketing strategy, brand identity, and business goals. Such an initiative would require extensive planning and thought leadership; it would not be undertaken as a reactive measure, such as for campaign or aesthetic usage.

Fanart

The GitLab brand has been a source of creative inspiration for team members and the wider community since its inception. In many cases, team members may find themselves creating spinoff art inspired by our values, software, and tanuki logo.

Artwork that draws inspiration from or is based on GitLab's intellectual property ("Fanart") is not always an accurate representation of our brand, product, and company, and may dilute our brand or infringe on our intellectual property rights. To protect our brand, GitLab team members must adhere to the following guidelines:

Do's:

- Add an "unofficial fanart" label to the Fanart to clarify that the content is not official GitLab content. The text can be small, but make sure it's conspicuous enough that anyone seeing the Fanart will also notice the label.
- Only share Fanart you are authorized to share, e.g. Fanart that you yourself have created. If in doubt, please reach out to the Legal team in the legal Slack channel.
- Share only on the following approved platforms:
 - Internal GitLab Slack channels and private messages to friends.
 - You can share publicly to broader audiences from your personal social media accounts on Facebook, Twitter, Instagram, TikTok, or Reddit.

- For "public" shares:
 - LinkedIn sharing is prohibited due to the close association between one's personal LinkedIn profile and their workplace.
 - If you want to share Fanart publicly on any other platform not mentioned above, reach out in the brand Slack channel to discuss.

Don'ts:

- Do not create or share Fanart that uses the GitLab logo or wordmark.
 - If your Fanart incorporates any other GitLab IP (such as screenshots, likenesses, or imagery taken directly from GitLab materials), reach out to brand for review.
- Do not imply that the Fanart is official GitLab content. For example:
 - Do not use Fanart in any GitLab materials, such as slide decks, printed collateral, or swag.
 - Don't use or distribute Fanart to GitLab customers, or at events that you attend on GitLab's behalf.
- Do not sell or otherwise use Fanart for commercial purposes. For example:
 - Do not sell Fanart stickers or T-shirts.
 - Don't use Fanart to advertise any businesses, services, or products.
- Do not create or share any Fanart that is potentially offensive or otherwise detrimental to GitLab's brand or any other brand.
- Do not combine Fanart with other companies' materials or anything else unrelated to GitLab.

If you have questions, reach out to the Brand Creative and Brand Strategy teams in the brand Slack channel.

Canva best practices

Canva enables team members to self-service their design needs and create on-brand assets. Canva is a great tool for items that need a quick-turnaround or that have ever-changing content (like A/B ad testing). Canva supports print and digital designs, as well as video and animation; team members most frequently use it for digital ads and promotional items. For larger projects that require greater detail or conceptualizing, we recommend you submit a design request to the Brand Design team instead.

To get started in Canva, please complete the checklist below:

1. Request access to the GitLab Enterprise account.
1. Explore Canva's help center and YouTube channel for detailed tutorials (their Canva Pro playlist is a good place to start).
1. Read the best practices outlined below.
1. Watch our demo to orient yourself to the GitLab Enterprise account (must be logged into GitLab Unfiltered on YouTube to view).

GitLab Enterprise access

If you regularly create designs in Canva, we ask that you work in the GitLab Enterprise Canva Pro account instead of using a free or personal account. To gain access, please submit an access request issue. Once this is completed, someone from the Brand Design team will grant you access.

We have a limited number of seats available, so we encourage teams to consolidate the number of people on their team who need access. Note: Shared logins are prohibited.

Benefits of working in the GitLab Enterprise account:

- Full-functionality: working within our account gives you access to all of Canva's Pro features.
- Visibility: having all our assets organized in one account offers greater transparency and iteration.
- Workflow: collaborating is made easy with quick access to our brand assets, templates, and design team approvals.

Extra tips:

- Make sure you are logged in to your GitLab Enterprise account when in Canva, as it is easy to be logged into the wrong one if you have pre-existing logins.
- You can toggle between your designs (Projects) and the team's designs (GitLab Enterprise) on the left navigation menu.
- Contact the Brand Design team if you need to change your permissions in Canva. Team members who join the account are automatically set to member status, which allows editing and sharing of files.

Files and folders

Folders can contain Canva design files, sub-folders, and assets you upload from your device. Once you create a folder, it will default to your folders under Projects. You must manually share the folder with GitLab Enterprise in order for it to be seen and/or edited by the rest of our organization.

- Select Projects from the navigation menu on the left > Hover over your Folder > Click the three dots > Share > Select the eye icon (view access) or pencil icon (edit access) from the GitLab Enterprise drop down

Designing in Canva

Templates

- Canva has a multitude of its own templates, but it is best to refrain from using these. They require more work to re-design, and they introduce branding that doesn't follow our guidelines.
- The templates in our GitLab Enterprise account are a great starting point for creating on-brand designs. We have templates in all ad sizes, in addition to other promo and resources.
 - Note: Our templates are located in the Folders tab in the GitLab Enterprise account; this is so we can organize the templates in folders instead of the Templates tab, which lacks that structure.
- If you click on a template file, Use this template is shown in a purple button. Selecting this option will automatically create a copy that you can start designing in.
 - Note: Do not select Edit Original unless you have prior approval from the Brand Design team. This alters the template for everyone.
- Remember to rename your new design, remove any unused pages or instructions in the file, and move it to a relevant folder in the GitLab Enterprise account.

New files

- You can create new designs with custom sizes, move, and rename files.
- File names should be lowercase and use hyphens in place of spaces. Include the file dimensions in the file name. Avoid acronyms.

Pages

- A single Canva file can contain up to 200 pages, which is helpful if you are creating multiple assets that have the same size and are related.
- You can duplicate, delete, and name each page within a Canva file.
- Copy and paste keyboard shortcuts work for duplicating elements from one page to another, or even from one file to another.
- Easily resize your file if you are replicating the design in different dimensions. Resize the elements on your design accordingly, as they get shifted when the file is resized.

Brand kit

- The Styles tab on the left navigation menu will pull up our color palette and font (Inter) from the GitLab brand kit.
- The Logos tab will show our logo, which you can drag and drop into your design.

Typography

- Add text from either the Text or Styles tab; this will automatically populate text boxes for you with Inter and the appropriate font weight. These text boxes will still need to be formatted according to our typography guidelines, though:
 - Left-align all text.
 - Use GitLab Sans Bold for headlines and calls-to-action and use Regular for subheads and body copy.
 - Set the line spacing to 1.1 for headlines and 1.2 for all other text; keep the letter spacing at 0 for all copy.
 - For call-to-action styling, copy from an existing templates or reference the guidelines.
 - Make sure the background behind the text provides sufficient contrast for readability.
 - Use either White or Charcoal text for accessibility purposes.

Graphics

- You can search and add graphics from the Elements tab, but please use with discretion. This tab is great for finding basic shapes, lines, and image frames; beyond that, use graphics from the GitLab icon library to keep your design on-brand.
- All elements can be dragged and dropped into your design. Upload branded assets in the Uploads tab. The .png file format with a transparent background works best.
- For most Canva elements, you can adjust the color of the graphic to make sure you're using GitLab's color palette. Refer to the color guidelines and use the accent colors sparingly.
- You can also change the line weight of some Canva elements. To keep consistent with our illustration guidelines, a line weight of 1-2 works best.

Photography

- If you need to add photos into your design, it is best to use our approved images from our photo library instead of Canva's photography shown in the Elements tab.
- Reference our photography guidelines when selecting and placing imagery.
 - You can request for the Brand Design team to source images from Adobe Stock for you, if you need additional options.

Layout and alignment

- Use Canva's recommended margin spacing for your file; these display as magenta lines when you move elements around in your design.
- Dashed magenta lines will highlight if elements are aligned to one another.
- Items can be locked in place, grouped, aligned, flipped, rotated, scaled, and layered, using the tools in the tool bar.

Exporting

To export your design, select Share in the upper right corner > Download > select your file type > and click Download to have the assets downloaded to your device.

From the Share menu, you can also invite others to collaborate in the file with you.

Depending on your Canva permissions, you may be able to share the file as a template by selecting Template > Publish Template.

Requesting design approval

Always share your work for review. If you are working from an existing Canva template, you will see the option to request design approval in the top right of the file. If you are creating your own design, you can open a brand review issue and link your file. For expedited requests, please reach out in the marketing-design channel on Slack with a link to the issue or file.

How we work

Brand Design team structure

We are all brand designers, brand champions, critically thinking problem solvers, strategists, and teammates · leveraging each others' strengths while growing our collective knowledge and expertise. We work by following GitLab's values and using issues and epics to track our work.

Responsibilities

- Be lean and efficient: leverage everyone's strengths and expertise and trust one another to make the best decisions for the team and our brand.
- Wear many hats: have the flexibility to tackle a variety of tasks demanded by the role - all for the greater good of the team and company.
- Develop creative concepts: drive the creative direction of your individual projects. For large-scale projects, Adam and Luke are responsible for core concept development, which is then presented to the wider team for discussion, feedback, and refinement.
- Carry out the creative: bring a creative direction to life with touch-point collateral informed by the creative direction, all while managing your time accordingly.
- Advocate for the GitLab Brand: know, contribute to, and uphold our Brand Guidelines, and review materials from within and outside our team to preserve our brand's integrity.

- Know the tools: be proficient in the Adobe Suite and Figma (for designing), Mural and FigJam (for brainstorming), Canva (for creating self-service assets), and the Google Suite (for company-wide materials).

Design approvals

The team should feel empowered to make the best decisions possible for the GitLab brand while seeking both structured and direct feedback from the team as often as possible. Adam and Luke are here to make final decisions as needed, but do not need to be the bottleneck of progress.

Team workflow

- Team check-ins: We have two recurring team syncs: (1) Monday Brand Creative hours call where we catch up and talk about work for the upcoming week. (2) Wednesday Brand Creative hours call where we get feedback on projects and collaborate on topics that need to be addressed.
- Working in issues: All design requests should use our issue templates and include the mktg-status::triage, corporate-marketing, and design issue labels to show up on our team's triage board. Michelle, the Senior Creative Operations Manager on the team, triages work at the beginning of the week, and team members are able to assign work to themselves, too.
 - Note: Everyone can see their personalized board by selecting the drop-down option where it currently says "Brand Design TRIAGE Board" on the left of the search bar.
 - Note: Epics can be viewed on the Epic board.
- Collaborating together: We work together as a team by playing to each of our strengths. Work is usually triaged out to team members whose design skills fit the request, or we collaborate together on a project using our combined talents.
- Small projects: Smaller projects are typically assigned to one designer and completed within 2 weeks. Given the quick turnaround at GitLab, we work in small iterations, referred to as the MVC (minimal valuable change) approach.
- Medium and large projects: For larger projects, we assess the request and then propose a timeline. For campaigns or projects with a unique concept, we kick off a brainstorming session either async or on a cross-functional call with the larger team, or whoever has been assigned, and then we pitch the concept(s) to the stakeholders.
- Sharing work: Work should be shared early and often, either in our team calls or async in our private brand-design-team slack channel. Drafts are typically shared async with the stakeholder using the design tab in issues or as a screenshot in comments.

Saving work

We upload and pull work locally from the Brand Design repository. Confidential projects and large print files are stored in the team Google Drive, which is organized the same way as the repository. We also have a team Figma for collaborative work.

- The top-level folders of the repository are organized by project type; from there, they are broken down by category, and in some cases fiscal year.
- By default, all source files should be stored in the repository, along with smaller exported files (such as digital ads). For larger exports (such as print-ready files), please store and share them in the team Drive.
- For folder names, use lowercase, dashes instead of space, and do not include any special characters.
 - Example: field-marketing-events

File naming conventions

- Files should be named with the following convention: [project-type]-[project-name]-[asset-type]-[issue-number]-[asset-dimensions]-[fiscal-year]
 - Example: display-ad-banner-0842-160x600-fy25
- Please do not add character symbols in file name: % , / (periods are okay, only if necessary)
- If the vendor specs or project requires you to follow a specific naming convention, then that convention overrides ours. No need to rename files once delivered.
- File names should be lowercase and use hyphens in place of spaces. Include the file dimensions in the file name. Avoid acronyms.
- Both the Google Drive and repository save version history; for that reason, files can be saved over as edits to the file are made.

Using git and terminal

- Push and pull work frequently, at least once a day, to keep our work up-to-date. If you are working on a file that others may be in as well, communicate it with the team so that no one saves over someone else's work.
- To get started, set up a local clone of the brand-design repository to your machine. For your day-to-day work, here are the typical git commands for pulling and pushing work, in the order of which they should be used:
 - cd brand-design - this command only needs to be used once upon opening the Terminal app. Depending on where you've stored the repository on your machine, the cd [parent folder name] command may need to be done multiple times, until you get into the brand-design folder
 - git pull - this updates your local repository to mirror any changes made by the rest of the team; do this command before starting work or pushing any new work
 - git status - optional command; this provides a summary of all the changes you've made locally that need to be pushed back the repository
 - git checkout [insert file path] - optional command; this can be used to remove any files you do not want to push to repository
 - git add . - use this command before pushing your work; it will add all the files you have made changes to
 - git commit -m "[insert description of changes]" - include a message with a summary of your changes; this is viewable to everyone and provides context
 - git push origin main - this will push all your changes back to the repository, including the commit message to describe the changes
 - git pull --rebase, followed by, git push origin main - use these two commands to reset if you receive an error when pushing

eBooks, whitepapers, and one-pagers

- Design templates for each publication are located here.
- Our publications are often sent to Smartling for translation; save out a .zip file of the packaged artwork for easy download.
- Most one-pagers can be self-served using one of our Google Doc templates.

Digital ads

- Digital ads often need to be customized after creation, and for that reason we encourage

Canva for easier self-service. Some ads need to be edited by Demandbase under certain specs and file requirements.

Events

- Trade Show booths should default to using our general GitLab brand. General events such as these are organized by year within the repository.
- Recurring events that we host (Sales Kickoff, Contribute, Commit) have a different theme each year, which inspires a new creative direction. These events have their own folder, and are then organized within by year.

Brand Guidelines

The Brand Design team maintains the Brand Guidelines on design.gitlab.com (Pajamas). For our team's purposes, we can make updates to the guidelines using Web IDE.

To get started, follow the workflow outlined below and check out our demo with the Product team.

Using Web IDE

1. Open Pajamas and navigate to the page you need to make updates to.
1. Scroll to the bottom of the page and select Open Web IDE.

Pajamas structure

Our Brand Guideline pages are located in the Contents folder and organized accordingly:

1. brand: This folder contains the Brand Guidelines Overview page.
1. brand-design: This contains pages related to design guidelines.
1. brand-logo: This contains pages relating to logo guidelines.
1. static > img > brand: The brand folder is where all media will be stored that links back to the Brand Guidelines.
1. nav.json: This file dictates the structure reflected in the navigation bar on Pajamas.

Creating a new page

1. Right click on the folder where the page will be located. Select New File.
1. Add your file name (this is what will be reflected in the URL path). End with the .md extension to denote a new markdown page.
1. Update the navigation bar with your new page by opening the nav.json file.
1. Follow the navbar configuration to organize your pages. Nesting pages below other pages will create dropdowns in the navigation bar.
 1. title is how you want your name to display on the page.
 1. path is what is reflected in the URL path (this should match the file name of the page you made).

Note: Pajamas uses markdown and the heading styles match the handbook. Add content as you normally would.

Optimizing and uploading media

1. .svg format is ideal because it scales with the page. Optimize your file before uploading by using SVGO.
1. .png or .jpg work well for graphics that contain images. Optimize your graphic before uploading.
1. .gif format is not accepted.
1. Upload all files to static > img > brand folder.
1. Videos can be embedded (either from Vimeo or YouTube).

Adding media

Add media using figures, which connect the visual with a caption. Here is the breakdown of their structure:

1. aria-label= This text can be the same as the fig caption.
1. img class= This formats the image's display size. img-50 scales the width down to 50% of the page. gl-p-5 scales to the full width of the page.
1. src= This corresponds with the image's location. This should match the file name that corresponds with what you've uploaded to the static > img > brand folder.
1. alt= This is alternate text that displays in the case that the media does not populate on the page; this should be more descriptive and unique from the aria-label and fig caption.
1. fig caption This is the descriptive text displayed below the graphic.

Pushing your merge request

Smaller commits make collaboration easier.

1. Use conventional commits. The title of your commit can be generic, and the merge request description can be more detailed. Type:feature covers most of the updates our team makes.
 1. type:feature: Effort to deliver new features, feature changes & improvements.
 1. type:maintenance: Up-keeping efforts & catch-up corrective improvements that are not Features nor Bugs.
 1. type:bug: Defects in shipped code and fixes for those defects.
 1. Commit example: feat(Br